

MAF220 — Numerical Modelling

Computer Lab, Fall 2026

Exercise 3a:

In the lectures, you have learned about Gauss Elimination, Gauss-Jordan, and LU decomposition methods to solve systems of linear equations in the form $Ax = b$. All these algorithms have a complexity of $\mathcal{O}(N^3)$, where N is the number of linear equations to solve (thus, A is an $N \times N$ matrix). Let's check this complexity empirically by solving linear systems with various sizes and measuring the computation time!

Warm-up

1. Create a 10×10 random matrix A and a 10-element random vector b . (Use `np.random.rand(a,b)` for an $a \times b$ matrix and `np.random.rand(a)` for an a -element vector.)
2. Solve the matrix equation $Ax = b$ with `linalg.solve(A, b)`.

Computational complexity: Analyse the complexity of the `linalg.solve` algorithm by measuring the execution time for increasing values of N , the dimension of the system (above, N was 10).

3. Choose an appropriate range of values for N , for example from $N = 2000$ to $N = 24000$, using a suitable step size (e.g., 2000). Use `np.arange` or `np.linspace` to generate this array.

Note: The exact values of N should be chosen based on the performance of your own computer (these values worked fine for mine). The goal is to reach sufficiently large matrix sizes to observe the growth in computation time, without making your machine unresponsive. You may need to adjust the range depending on how fast or slow your system is.

4. For each N , create a random matrix A of shape $N \times N$ and a random vector b of length N using `np.random.rand`.
5. Measure the execution time using the `time` module:

```
import time

start = time.time()

# code to be measured

end = time.time()
elapsed_seconds = end - start
```

This method gives a rough estimate of the runtime, which is sufficient for the purpose of this experiment.

6. Use `np.linalg.solve` to solve the linear system $Ax = b$ and record the execution time for each value of N .
7. Plot the results on a **log-log scale**: plot $\log(N)$ on the x-axis and $\log(\text{time})$ on the y-axis.
8. Fit a straight line to the log-log data (e.g., using `np.polyfit`) and include it in the plot. The slope of this line corresponds to the exponent p in the relation:

$$T(N) \sim k \cdot N^p$$

This tells you how the runtime scales with matrix size.

Note: The constant k reflects the baseline speed of the algorithm on your specific machine. It depends on factors such as your CPU, how efficient the linear algebra libraries are (e.g., MKL, OpenBLAS), and whether the computation is multithreaded. This is why timing results can differ significantly from one computer to another, even if the scaling exponent p is similar. **Remark:** The cubic complexity $\mathcal{O}(N^3)$ is an *asymptotic* property — it describes how the algorithm behaves as N becomes very large. For small or moderate values of N , execution time may not follow a perfect cubic trend due to other dominating factors such as:

- Function call and setup overhead
- Memory access latency and CPU cache behavior
- Optimization techniques used by the underlying libraries

These effects diminish for larger N , where the N^3 behavior becomes more apparent.

9. **Extra:** You might wonder why the randomly initialized matrices A do not happen to be singular. Can you think of a reason why it is that way?

The goal of this exercise is to illustrate that matrix operations become expensive very quickly! $\mathcal{O}(N^3)$ is a very bad complexity. There exist faster, more complicated algorithms for solving systems of linear equations, e.g., the Strassen algorithm, which scales as $\mathcal{O}(N^{2.8074})$.

Exercise 3b:

Although LU decomposition has the same complexity as Gauss Elimination or Gauss-Jordan, it is advantageous if a linear system (one particular matrix A) needs to be solved for multiple vectors b . The reason is that the expensive part is the decomposition of A into the matrices $(P)LU$. Given this decomposition, the solution of the matrix equation can be obtained with backward substitution, which has a reduced complexity of $\mathcal{O}(N^2)$.

Let's explore that! SciPy (an even more powerful Python module than NumPy; import it via `import scipy`) offers a method for the more general PLU decomposition:

$$A = PLU \tag{1}$$

P ...Permutation matrix

L ...Lower triangular matrix

U ...Upper triangular matrix

A permutation matrix is a matrix with only one value of one per row, all others are zero. If applied to a matrix, it simply permutes the rows of the matrix. This procedure is called pivoting and is often done in numerical algorithms to reduce round-off errors. As in the previous exercise, we are going to randomly initialize matrices A . To get a trivial permutation matrix (a unity matrix) in the PLU decomposition, we are going to solve the linear equation with the modified matrix $\tilde{A} = P^{-1}A = LU$.

Warm-up

1. Create a 10×10 random matrix A and a 10-element random vector b . (Use `np.random.rand(a,b)` for an $a \times b$ matrix and `np.random.rand(a)` for an a -element vector.)
2. Set up a matrix $\tilde{A}$, such that P is a unity matrix in the following PLU decomposition.
 - Calculate the PLU decomposition with `P, L, U = scipy.linalg.lu(A)`.
 - Calculate $\tilde{A} = P^{-1}A = LU$.
3. To solve the matrix equation $\tilde{A}x = b$ with the LU decomposition method (because of our preparations above, P is unity and we can forget about it), we first need the decomposition `P, L, U = scipy.linalg.lu(A_tilde)`.

4. Given the decomposition, we need to solve the following two equations with substitution to obtain the result x :

$$Ly = b \tag{2}$$

$$Ux = y \tag{3}$$

5. Do not use the `np.linalg.solve` method! We need to tell the computer about the special form of the matrices so that it can apply the efficient back-substitution algorithm: Use the `scipy.linalg.solve_triangular` method from the SciPy package. Read the SciPy documentation for the significance of the flags "lower" and "unit diagonal" (google is your friend!)
6. If you want, you can check if the result is the same as the `np.linalg.solve` method gives you.

Computational complexity: Analyse the complexity of the algorithm above by measuring the execution time for an increasing number of times the matrix equation needs to be solved. Compare it to the `np.linalg.solve` method. Use $N = 1000$.

7. Create a random matrix A of size $N \times N$ (use `np.random.rand`).
8. Set up a matrix $\tilde{A}$, as explained above.
9. Create a list or an array that contains the number of times M the system should be solved, starting from 10 up to 100 in step size 10. (Use `np.arange`.)
10. Now measure how long it takes to solve the system $\tilde{A}x = b$ with the LU decomposition method M times. (Generate a new random vector b every time.) Store how long it took for each value of M (also add the time the LU decomposition took!).
11. Repeat the last step using the `np.linalg.solve` method.
12. Make a plot that compares the execution time for both methods with an increasing number of M .
13. Compare the measured times in the plot: Is the LU decomposition faster? Is the speed-up increasing with the value of M ?